Ten Brief Lessons on Web Programming

A concept notebook on web programming

"There are 10 types of people in the world:
those who understand binary, and those who don't."

Which side are you on?

Giuseppe Costanzi

This page is intentionally left blank.

Ten Brief Lessons on Web Programming
A concept notebook on web programming

Copyleft © 2026 Giuseppe Costanzi *aka* `1966bc`

Version 1.0 — First edition, MMXXVI AD / MMDCCLXXIX *ab urbe condita*

This book is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Source code: `https://github.com/1966bc/bibliotheca`

Built with pure PHP, JavaScript, HTML, CSS and SQLite.
Typeset with LaTeX via Pandoc.
No frameworks were harmed in the making of this book.

`giuseppe.costanzi@gmail.com`

Milky Way Galaxy, Solar System, third planet from the Sun
41°53'35"N 12°28'58"E — Roma, Italia

To the next generation of programmers:
learn to read, write and compute —
before it's too late.

Contents

Who Is This Book For **8**
What you will need . . . 8
What you will learn . . . 8
What this book is not . . . 8

0 Prelude **9**
0.1 What is this . . . 9
0.2 Who is this for? . . . 9
0.3 How to read this . . . 9
0.4 What you will need . . . 9

1 Introduction **11**
1.1 The Shape of Things to Come . . . 11
1.2 Some useful principles to guide our decisions . . . 11
1.3 Coding standards . . . 11
1.4 Our tools, the stack . . . 12
1.5 The terminal . . . 12
1.6 The body of a web application . . . 12
1.7 Architecture . . . 13

2 Database **14**
2.1 Why start here? . . . 14
2.2 SQLite . . . 14
2.3 The domain . . . 14
2.4 Schema conventions . . . 14
2.5 The schema . . . 15
2.6 Data types . . . 15
2.7 Setting up the console . . . 15
2.8 File structure . . . 15

3 Project Structure **17**
3.1 Security by design . . . 17
3.2 The directory tree . . . 17
3.3 Two worlds . . . 18
3.4 The Stargate . . . 18
3.5 The journey of a request . . . 18

4 Backend **20**
4.1 Prerequisites . . . 20
4.2 DBMS — the database class . . . 20
4.3 PHP in the MVC . . . 21
4.4 The Model . . . 22
4.5 The Controller . . . 22
4.6 Testing with curl . . . 22

5 Frontend **23**
5.1 The flow . . . 23
5.2 Routing . . . 23
5.3 Pages: plural and singular . . . 23
5.4 fetch: the bridge . . . 24
5.5 DOM: building the page . . . 24
5.6 The JavaScript classes . . . 25
5.6.1 Table pages . . . 25
5.6.2 Form pages . . . 26

6 CRUD **28**
6.1 The sign of the four . . . 28
6.2 Soft delete vs hard delete . . . 29
6.3 From form to database . . . 29
6.3.1 Create, POST . . . 29
6.3.2 Read, GET . . . 29
6.3.3 Update, PUT . . . 29
6.3.4 Delete, DELETE . . . 30

7 Validation **31**
7.1 Three lines of defense . . . 31
7.2 What we validate in our project . . . 31
7.3 Input normalization . . . 32
7.4 Sanitization . . . 32
7.5 Error display: how do we tell the user something is wrong? . . . 32
7.6 Server errors . . . 33
7.7 Exceptions: who catches what . . . 33

8 Permissions **35**
8.1 The problem . . . 35
8.2 How Linux permissions work . . . 35
8.3 The fix . . . 35
8.4 Every time you recreate the database . . . 36
8.5 Why `public/` matters . . . 36

9 Debugging **37**
9.1 The blank page . . . 37
9.2 Think in layers . . . 37
9.3 Apache error log . . . 37
9.4 curl: testing the API . . . 38
9.5 Browser developer tools . . . 38
9.6 PHP debugging tools . . . 38
9.7 PHP from the command line . . . 39
9.8 SQLite from the command line . . . 39
9.9 The golden rule . . . 40

10 Security **41**
10.1 What we already have . . . 41
10.2 CSRF — protecting write operations . . . 41
10.2.1 The attack . . . 41
10.2.2 The defense: a secret token . . . 41
10.2.3 The Csrf class . . . 42
10.2.4 Verify it works from the terminal . . . 42
10.3 Sessions and session fixation . . . 43
10.4 Security headers . . . 43
10.4.1 Content-Security-Policy . . . 44
10.5 Rate limiting . . . 44
10.6 HTTPS . . . 45
10.6.1 How it works . . . 45
10.6.2 Where it lives . . . 45
10.6.3 Getting a certificate . . . 46
10.7 The layers . . . 46
10.8 The rule . . . 46

A How It Works **47**
A.1 Request and response . . . 47
A.2 Open your terminal . . . 47
A.3 The first request . . . 47
A.4 Where are the records? . . . 48
A.5 Two requests, one page . . . 48
A.6 What travels on the wire . . . 49
A.7 What Apache does . . . 49
A.8 What the browser does . . . 50
A.9 What JavaScript does . . . 50
A.10 How data changes shape . . . 51
A.11 The chain . . . 51
A.12 What happens when you click Save . . . 52
A.13 What `fetch` sends to the controller (`api/categories.php`) and what it gets back 53
A.14 Inside the controller (`api/categories.php`) . . . 53
A.15 Below the surface, under the browser's surfing: `src/` . . . 55
A.16 What comes back . . . 56
A.17 Edit mode . . . 57
A.18 The complete picture . . . 57

B Glossary **58**

C Bibliography **62**
C.1 The terminal . . . 62
C.2 Apache . . . 62
C.3 PHP . . . 62
C.4 JavaScript, HTML, CSS . . . 62
C.5 SQL and SQLite . . . 62
C.6 HTTP . . . 62
C.7 Git . . . 62

C.8 The classics 63
C.9 Security 63

D The Apocryphal Chapter — Beyond the Launch 64
D.1 What you actually learned 64
D.2 What Bibliotheca does not do 64
D.3 Frameworks — when and why 65
D.4 The age of AI 65
D.5 42 66

Who Is This Book For

For the young programmer — or the curious one at any age — who wants to understand how a web application actually works, in anno Domini 2026. From the database to the browser, with nothing in between but your own code.

You do not need experience with web development. But you should be comfortable with basic programming: variables, functions, loops, conditionals. You should know what a class is and how objects work — we use Object-Oriented Programming throughout, and we will not stop to explain inheritance or encapsulation.

If you have never written a line of code, start there first. Python is a good choice. Then come back.

What you will need

- A **Linux** machine (we use Debian, but any distribution works)
- **Apache**, **PHP** and **SQLite** installed
- A **text editor** (we recommend Sublime Text)
- A **terminal** and a **web browser**
- **Curiosity**

What you will learn

- How HTTP works — requests, responses, status codes
- How to design a relational database with SQLite
- How to structure an application with Model-View-Controller
- How to build a REST API with pure PHP
- How to build a dynamic frontend with pure JavaScript
- How to implement CRUD operations from click to database row
- How to validate, sanitize, and normalize user input
- How to handle permissions, soft deletes, and dependencies
- How to debug when things go wrong
- How to secure your application: authentication, CSRF, headers, HTTPS

What this book is not

This is not a reference manual. It is a journey. Each chapter builds on the previous one. Read in order. Type the code yourself. Break it. Fix it. That is how you learn.

We use no frameworks, no libraries, no build tools. Not because they are bad — but because you need to understand what they replace before you use them.

Chapter 0

Prelude

0.1 What is this

The best way to learn something is by imitation. Study a real application, understand how it works, then build your own.

Bibliotheca is a complete web application — a book catalog. Simple enough to hold in your head, rich enough to teach you the fundamentals of web development.

The entire source code is on GitHub: https://github.com/1966bc/bibliotheca

Read it. You will read more code than you write in your career — and reading good code is how you learn to write it. Reading *why* it is written that way is even better.

0.2 Who is this for?

For the young programmer's mind — or the curious one at any age — open to the wonder of understanding how things actually work. A web application, in anno Domini 2026. From the database to the browser, with nothing in between but your own code.

You do not need experience with web development, but you should know the basics of programming: variables, functions, loops, conditionals. You should also have some intuition of how a website works: that a browser asks a server for something, and the server answers. And above all, the hacker's instinct: the urge to open things up and see how they work.

0.3 How to read this

Read in order and try to absorb the key concepts. The lessons are brief but there is a lot to take in. Look at the application as you go — piece by piece, the puzzle will come to life.

0.4 What you will need

- A **Linux** machine (we use Debian 12 Bookworm, but any distribution works)
- A **LAMP** stack: Linux, Apache, MySQL/MariaDB, PHP — even though we use SQLite instead of MySQL, Apache and PHP are essential. Install them with:

```
sudo apt install apache2 php libapache2-mod-php php-sqlite3
```

- A **text editor** — we recommend Sublime Text, fast and lightweight. You do not need a heavy IDE to write good code.

- A **terminal**
- A **web browser**
- **Curiosity**

One note about the terminal. Throughout this book, we will use the command line for everything we can: creating files, running queries, testing the API, checking permissions, managing git. This is deliberate. The terminal is not a fallback for when the GUI is missing. It is the primary tool of a programmer. Learn to think in commands.

One more thing. A programmer must have dominion over the code. Every line, every layer, every file — you must know what it does and why it is there. If you do not understand it, you do not control it. And if you do not control it, sooner or later it controls you.

Let the show begin.

Chapter 1

Introduction

1.1 The Shape of Things to Come

Projects begin in the mind, not on a computer. Then they take shape with pen on paper. Because before writing a single line of code, we need to define our rules and our domain. This is how real projects are built.

As your thoughts take shape, write them down in your own words. Reading and writing are two different cognitive processes. Reading lets you follow someone else's reasoning. Writing forces you to build your own. You will be surprised by the gap between "I understood it" and "I can explain it." That gap is where learning happens. Otherwise you end up like Augustine of Hippo: *"If no one asks me, I know; if I wish to explain it to one who asks, I do not know."*

1.2 Some useful principles to guide our decisions

- **Böhm-Jacopini theorem** — Sequence, selection (if/else), iteration (for/while). No magic, no tricks.
- **KISS** — Keep It Simple, Stupid. Always the simplest solution that works.
- **DRY** — Don't Repeat Yourself.
- **YAGNI** — You Aren't Gonna Need It. Don't build what you don't need yet.
- **SoC** — Separation of Concerns. Each part of the code has one job.
- **Least Surprise** — Code should behave as the reader expects.
- **Fail Fast** — If something goes wrong, stop immediately.
- **Fail Safe** — When you fail, fail securely.

1.3 Coding standards

We follow consistent conventions throughout the project:

- **Indentation**: 4 spaces everywhere (PHP, JavaScript, HTML, CSS, SQL). No tabs.
- **Classes**: `PascalCase` — `Book`, `PublishersView`, `CategoryForm`.
- **Methods and variables**: `camelCase` — `getAll`, `findById`, `bookTitle`.
- **Constants**: `UPPER_SNAKE_CASE` — `DB_PATH`, `API_URL`.
- **Files**: `snake_case`, lowercase. Plural for collections (`publishers.php`), singular for single entity (`publisher.php`). PHP class files match the class name (`Book.php`).
- **SQL**: keywords `UPPERCASE`, tables and columns `snake_case`. Always prepared statements with named parameters (`:id`, `:title`).
- **PHP**: `declare(strict_types=1)` at the top of every file. Opening brace on the same line for control structures, next line for classes and methods.
- **JavaScript**: `'use strict'` at the top. Always `const` or `let`, never `var`. Always `async/await`, never `.then()` chains. Semicolons required.

- **HTML**: double quotes for attributes, `kebab-case` for IDs and classes, semantic tags.
- **Comments**: English only. Explain *why*, not *how*.

1.4 Our tools, the stack

Layer	Technology	Notes
Backend	PHP (pure)	No frameworks
Frontend	JavaScript (pure)	No frameworks
Markup	HTML (pure)	No template engines
Style	CSS (pure)	No preprocessors
Database	SQLite	Via PDO

Pure languages only. We use nothing but the native capabilities of each language. No extra tools to learn, no abstractions to cloud your understanding. Stay pure, stay focused.

1.5 The terminal

There is one tool that sits above all others: the command line. Every operation in this book can be done from the terminal, and most of them *should* be. Creating a database, inserting seed data, testing an API endpoint, checking file permissions, committing code — all from a text prompt.

Why? Because a graphical interface hides what is happening. A command shows it. When you type `chmod 664 bibliotheca.db`, you know exactly what you are changing. When you right-click and open a properties dialog, you are trusting someone else's abstraction.

More importantly: a production server has no desktop, no file manager, no mouse. It has a terminal and nothing else. If you can only work with a GUI, you cannot work on a real server. The programmers who are comfortable on the command line are the ones who can diagnose a problem at 2 AM on a server they have never seen before.

The command line has been the programmer's primary interface since the DEC VT100 in 1978. Its screen had 80 columns and 24 rows — and that is where the 80-character line convention comes from, the same convention you still see in coding standards today. Graphical desktops came and went. The terminal is still here, because it works. In programming, traditions exist for a reason.

Make the terminal your first choice, not your last resort.

Each of these technologies has extensive documentation online. Study them separately. This book shows how the pieces fit together. You will find all references in the Bibliography appendix.

1.6 The body of a web application

Let us begin with an abstraction. Before reading about architecture, look at your own body.

The skeleton (HTML) — Strip everything away: skin, clothes, muscles. What remains is structure. A skull, a spine, ribs. HTML is this: it says *what exists* — a heading, a table, a form,

a navigation menu. A website with no CSS and no JavaScript still works. Try it: disable both in your browser. The page is ugly, naked, but readable. That is the skeleton test.

The skin (CSS) — Now dress the skeleton. Skin gives visible shape to bones. Clothes add style, color, identity. CSS does exactly this. But if you change clothes, the bones underneath do not change.

The nervous system (JavaScript) — Bones and skin alone are a statue. Beautiful, but still. The nervous system carries signals: from senses to brain, from brain to muscles. JavaScript moves data: it takes the user's click on "Save", transmits it to the server, receives the response, and updates the skeleton. It does not decide *what* to show or *how* it looks. It connects and transports.

The internal organs (PHP + Database) — The heart pumps, the liver filters, the kidneys purify. You cannot see them, but without them the body dies. The PHP backend and the SQLite database receive requests, validate data, save it, retrieve it, protect its integrity. The user does not know they exist — and that is how it should be.

When layers mix — HTML inside JavaScript, SQL inside HTML — it is like a body with the liver attached to the knee. Does it work? Maybe. Is it healthy? Never.

1.7 Architecture

Model-View-Controller (MVC), simplified for clarity. Now you can map the body metaphor to the technical terms:

- **Model** — The organs. PHP classes that talk to the database and return data.
- **Controller** — The nervous system. PHP scripts (API) that receive requests and respond with JSON, plus JavaScript that carries data to the interface.
- **View** — The skeleton and skin. HTML structure + CSS appearance.

Chapter 2

Database

2.1 Why start here?

The database is the foundation. If the schema is solid, everything else follows naturally:

Remember MVC?

From tables come Models. From Models come Controllers. From Controllers come Views.

In the beginning was the table.

2.2 SQLite

We chose SQLite for its simplicity. No server to install, no users to configure, no permissions to manage. One file and you are ready.

PDO (PHP Data Objects) works the same way regardless of the database engine. If one day you need to switch to MySQL or PostgreSQL, you change the connection string and nothing else.

2.3 The domain

Bibliotheca is a book catalog. So the domain has four entities:

- **Publisher** — who publishes the book (Einaudi, Penguin Books. . .)
- **Category** — the classification (Fiction, Science, Philosophy. . .)
- **Author** — who writes the book
- **Book** — the core entity

And their relationships:

- A book has **one** publisher (many-to-one)
- A book has **one** category (many-to-one)
- A book can have **many** authors, an author can have **many** books (many-to-many, via the `book_author` junction table)

2.4 Schema conventions

Every table follows the same structure, a convention we impose on ourselves:

1. First field: primary key (`table_id`)

2. Second field(s): foreign keys (if any)
3. Data fields
4. Last field: `status` (soft delete — 1 active, 0 inactive)

Take note: we always access data **by column name**, never by position. Say it out loud. You may not fully understand why yet, but one day you will be grateful for this choice.

2.5 The schema

```
publisher (publisher_id, name, status)
category  (category_id, name, status)
author    (author_id, first_name, last_name, birthdate, status)
book      (book_id, publisher_id, category_id, title, pages, published, status)
book_author (book_id, author_id)
```

Five tables. Four entities, one junction table. Simple enough to hold in your head while reading the code.

2.6 Data types

The schema intentionally covers the fundamental data types:

- **TEXT** — strings (title, name, first_name, last_name)
- **INTEGER** — numbers (pages, status, primary and foreign keys)
- **TEXT as DATE** — dates in ISO 8601 format (published, birthdate)

2.7 Setting up the console

SQLite has a command-line interface. A programmer should always prefer the command line. It builds character, and keeps you away from the dark side of the force. We configure it with a `setconsole` file that sets headers, column mode, and enables foreign keys:

```
sqlite3 bibliotheca.db -init sql/setconsole
```

2.8 File structure

```
sql/
    setconsole           SQLite console configuration
    run.sql              scratch pad for quick queries
    ddl/
        create_table.sql   the schema (CREATE TABLE statements)
    dml/
        insert.sql         sample data (INSERT statements)
    dql/
        (queries will go here)
    bibliotheca.db       the database file
```

- **ddl/** — Data Definition Language (structure)
- **dml/** — Data Manipulation Language (data)
- **dql/** — Data Query Language (queries)

Take your time in this folder. Read the files, chew on them. It will pay off.

Chapter 3

Project Structure

3.1 Security by design

The most important decision in a web application structure is what the browser can and cannot reach. Everything inside the document root is accessible via URL. Everything outside is not.

This is not a feature. It is a security rule.

3.2 The directory tree

```
bibliotheca/
    docs/                       the book you are reading
    sql/                        database (schema, data, queries)
        ddl/                    CREATE TABLE statements
        dml/                    INSERT statements
        dql/                    SELECT queries
        setconsole              SQLite console configuration
        run.sql                 scratch pad for quick queries
        bibliotheca.db          the database file
    src/                        PHP classes  NOT accessible from the web
        DBMS.php                database wrapper
        Publisher.php           Publisher model
        Category.php            Category model
        Author.php              Author model
        Book.php                Book model
    public/                     document root  ONLY folder exposed to the browser
        .htaccess               URL rewriting rules
        index.php               the router
        css/
            style.css
        js/
            publishers.js       list view (plural)
            publisher.js        form view (singular)
            ...                 same pattern for each entity
        api/
            publishers.php      JSON endpoint
            ...                 one per entity
        pages/
            home.php
            publishers.php      list page (plural)
            publisher.php       form page (singular)
            ...                 same pattern for each entity
```

```
        404.php
```

3.3 Two worlds

- **Public** (`public/`) — what the browser can reach. HTML, CSS, JavaScript, and the API endpoints.
- **Private** (`src/`, `sql/`) — what only PHP can reach.

3.4 The Stargate

There is a principle in programming: don't talk to strangers. The Stargate is where we enforce it. Everything outside `public/` is invisible to the browser. The gate decides who passes and where they go.

When the browser requests `/publishers`, Apache looks for a file or directory called `publishers` inside `public/`. It does not exist. Before giving up with a 404, Apache checks if there is an `.htaccess` file in that directory with instructions. It finds ours, reads it, and follows the rewrite rule written in it:

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?route=$1 [L,QSA]
```

Line by line:

1. **RewriteEngine On** — enable URL rewriting.
2. **RewriteCond !-f** — only rewrite if the request is NOT an existing **f**ile (so `style.css` and `book.js` are served normally). The `-f` stands for *file*.
3. **RewriteCond !-d** — only rewrite if the request is NOT an existing **d**irectory. The `-d` stands for *directory*.
4. **RewriteRule** — capture the entire URL path and pass it to `index.php` as the `route` parameter. `[L]` means stop processing rules. `[QSA]` means append any existing query string (so `/book?id=3` becomes `index.php?route=book&id=3`).

The result: the user sees clean URLs (`/publishers`, `/book?id=3`), but every request actually goes through `index.php`. This technique is called **routing**: one entry point, many destinations. If a route is not in the whitelist, it does not pass. It is the foundation of all modern web applications.

Note: `.htaccess` requires Apache's `mod_rewrite` module, which is the engine that makes URL rewriting possible. If clean URLs do not work, enable it with `sudo a2enmod rewrite` and restart Apache.

3.5 The journey of a request

```
Browser                          Server
```

```
URL: /publishers
   .htaccess rewrites to index.php
         loads pages/publishers.php
               loads js/publishers.js

                      fetch('/api/publishers.php')

                     api/publishers.php (Controller)

                                                 require '../../src/Publisher.php'

                                                src/Publisher.php (Model)

                                                         PDO query

                                                        sql/bibliotheca.db
```

Chapter 4

Backend

In our architecture, PHP has one job: talk to the database and return data. It lives outside `public/`, invisible to the browser. It receives requests, processes them, and answers with JSON. The internal organs, remember?

Most PHP books and tutorials embed PHP directly inside HTML. Do not let that mislead you. PHP's real job is the backend: validate, query, respond. For everything the user sees and interacts with, there is JavaScript. This decoupling is fundamental. The sooner you internalize it, the better.

4.1 Prerequisites

PHP needs the SQLite driver. Check if it is installed:

```
php -m | grep -i sqlite
```

If empty, install it and restart Apache:

```
sudo apt install php-sqlite3
sudo systemctl restart apache2
```

4.2 DBMS — the database class

Open `src/DBMS.php` and read it. This is the only class that talks to the database. Every query, every insert, every transaction passes through it. It is the first file you should study.

Most tutorials have you open a connection, write the query, and fetch results on every single page. That is not how it works in real projects. We write it once, in one place. Everyone else uses it. That is DRY in action.

Naturally, it is a class we wrote ourselves. A thin wrapper around PDO. It exposes two ways to execute SQL, matching what PDO itself offers:

- **`query()`** — for fixed SQL with no parameters. Example: `SELECT * FROM publisher ORDER BY name`.
- **`fetchOne()` / `fetchAll()`** — for SQL with named parameters, using prepared statements. Example: `WHERE publisher_id = :id`.

And convenience methods for write operations (all via prepare): `insert()`, `update()`, `delete()`.

It also provides `exec()` for statements that return no rows (DDL, PRAGMA), and `beginTransaction()` / `commit()` / `rollBack()` for atomic operations — when two or more statements must succeed together or fail together.

As a safety net, `query()` and `exec()` reject SQL that contains named parameters (`:name`). If you need parameters, use prepared statements — no exceptions.

Key choices in the constructor:

- `ERRMODE_EXCEPTION` — fail fast on errors.
- `FETCH_ASSOC` — access data by column name, never by position.
- `PRAGMA foreign_keys = ON` — SQLite ignores foreign keys by default.

4.3 PHP in the MVC

Remember MVC from Chapter 00? PHP lives in the Model (`src/`) and in the Controller (`public/api/`). JavaScript and HTML/CSS handle the rest.

4.4 The Model

Open `src/Publisher.php`. The Model receives the DBMS in its constructor (**dependency injection**) and provides methods to query one entity. It does not know where the database lives or how it was opened. It just uses it.

The `status` column (1 = active, 0 = disabled) controls visibility. `getAll` returns everything; `getActive` filters by `status = 1`. Disabled records appear greyed out in the list. Deletion is permanent (`DELETE FROM`).

4.5 The Controller

Open `public/api/publishers.php`.

The Controller is the only PHP file the browser can reach. It creates the DBMS and the Model, reads the HTTP request, calls the Model, and returns JSON. Nothing else.

4.6 Testing with curl

PHP runs on the server. Unlike JavaScript, it does not need a browser. Keep these two worlds separate in your mind. As you grow, you will notice subtle exceptions, but for now: PHP is the server, JavaScript is the browser. We can prove it right now from the command line:

```
curl http://localhost/bibliotheca/public/api/publishers.php
```


```
[{"publisher_id":7,"name":"Addison-Wesley","status":1},
 {"publisher_id":1,"name":"Adelphi","status":1},
 {"publisher_id":2,"name":"Einaudi","status":1},
 {"publisher_id":3,"name":"Feltrinelli","status":1},
 {"publisher_id":8,"name":"Gallimard","status":1},
 {"publisher_id":5,"name":"Laterza","status":1},
 {"publisher_id":4,"name":"Mondadori","status":1},
 {"publisher_id":6,"name":"Penguin Books","status":1}]
```


Eight publishers, ordered by name. The backend works.

Chapter 5

Frontend

5.1 The flow

1. The browser requests a URL (e.g., `/publishers`).
2. Apache rewrites it through `.htaccess` to `index.php`.
3. The router loads the right page from `pages/`.
4. The page includes a JavaScript file.
5. JavaScript calls `fetch` to the API, gets JSON back.
6. JavaScript builds the HTML from the data.

The HTML page loads once. After that, all data exchange happens through `fetch` calls that return JSON. The browser never asks the server for a new page.

5.2 Routing

All URLs pass through a single entry point: `index.php`. Apache makes this possible with one rewrite rule in `.htaccess`:

```
RewriteRule ^(.+)$ index.php?route=$1 [QSA,L]
```

Every request that does not match an existing file gets rewritten. The URL `/publishers` becomes `index.php?route=publishers`.

Inside `index.php`, the route is checked against a whitelist:

```php
$routes = ['home', 'login', 'publishers', 'publisher',
           'categories', 'category', 'authors', 'author',
           'book', 'about'];
```

If the route is in the list and the file exists, the corresponding page is loaded. Otherwise, the user gets a 404. No user input ever becomes a file path directly.

5.3 Pages: plural and singular

In our project, each entity has two pages, by convention:

- **Plural** (`publishers.php`): the list, with Edit and Delete buttons shown if the user is logged in.
- **Singular** (`publisher.php`): the form, for adding or editing.

One page, one job. Separation of Concerns.

5.4 fetch: the bridge

`fetch` is the browser's native API for HTTP requests. It replaced `XMLHttpRequest`, the old AJAX interface that powered the first generation of dynamic web applications. The idea is the same: JavaScript talks to the server without reloading the page. The API is simpler.

`fetch` is natively asynchronous: the browser sends the request and continues executing code without waiting for the response. This is necessary because network calls are slow compared to everything else, and blocking the browser would freeze the page.

The `async/await` syntax lets us write asynchronous code that reads like sequential code. Without it, we would need callbacks or promise chains. With it, each line waits for the previous one to finish, exactly as you would expect.

Reading data:

```
async load() {
    const response = await fetch('/api/publishers.php');
    const publishers = await response.json();
}
```

Three lines. Request, parse, done. No callbacks, no libraries. The `async` keyword on the method is required: `await` only works inside an `async` function.

Sending data requires a bit more: the HTTP method, headers, and a JSON body.

```
const payload = { name: 'Adelphi' };

const response = await fetch('/api/publishers.php', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'X-CSRF-Token': token
    },
    body: JSON.stringify(payload)
});
```

The payload is the data we want to send. `JSON.stringify` converts the JavaScript object into a JSON string, because HTTP only carries text. The CSRF token travels in the header. The data travels in the body. The server reads both before doing anything.

5.5 DOM: building the page

The DOM (Document Object Model) is the browser's internal representation of the HTML. Every tag becomes a node, every attribute a property. JavaScript does not modify the HTML file. It modifies this tree, and the browser redraws the screen.

Take this HTML from our publishers page:

```
<table id="publisher-table">
    <thead>
        <tr>
            <th>Name</th>
            <th></th>
```

```
        </tr>
    </thead>
    <tbody></tbody>
</table>
```

The browser reads it and builds the DOM, a tree of objects:

```
table #publisher-table
 thead
    tr
        th  "Name"
        th
 tbody  (empty)
```

JavaScript actually sees this tree, not the HTML text.

This is a crucial point. The `id` on the table and the empty `tbody` are placeholders. They tell JavaScript *where* to inject the data retrieved by `fetch`. The HTML defines the structure; JavaScript fills it. When we call `document.querySelector('#publisher-table tbody')`, we get the `tbody` node, ready to receive rows.

A concrete example from our code. When the book form loads, it populates a dropdown with publishers fetched from the API:

```
for (let i = 0; i < publishers.length; i++) {
    const option = document.createElement('option');
    option.value = publishers[i].publisher_id;
    option.textContent = publishers[i].name;
    this.selectPublisher.appendChild(option);
}
```

This is the core mechanism: JavaScript fetches the data, creates new HTML elements on the fly, places them in the right spot in the DOM, and can style them through classes or attributes. All of this happens without reloading the page. `createElement` makes the node. `textContent` sets its text. `appendChild` attaches it to the page. Three operations, always in this order.

Remember, never `innerHTML` with external data. That is an XSS vulnerability. `textContent` is safe by design: it treats everything as plain text, never as HTML.

5.6 The JavaScript classes

In our project each HTML page has a dedicated JavaScript class. Every class has a **constructor** that stores references to DOM elements and binds event listeners. The methods that follow depend on the type of page.

5.6.1 Table pages

The table pages (`Publishers`, `Categories`, `Authors`, `Books`) display all records. These have two methods:

- **`load()`**: fetches all records from the API.
- **`render(data)`**: builds the table rows from the data.

5.6.2 Form pages

The form pages (`Publisher`, `Category`, `Author`, `Book`) handle create, edit, and delete. These have more methods:

- **`checkEdit()`**: reads the `?id=` parameter from the URL. If present, the form is in edit mode and calls `load(id)`. If absent, the form is in create mode and stays empty.
- **`load(id)`**: fetches one record from the API.
- **`render(data)`**: populates the input fields with the fetched data.
- **`save()`**: validates the input, builds a `payload` object, then calls `fetch` with POST (create) or PUT (update).
- **`remove()`**: asks for confirmation, then sends a DELETE request.

Notice that `save` and `remove` use the same `fetch` call structure. The only thing that changes is the HTTP method:

Operation	Method
Create	`POST`
Update	`PUT`
Delete	`DELETE`

Every HTTP request carries a **method**: a word that tells the server what the client wants to do. Most introductory books only teach two: GET to read a page, POST to send a form. But HTTP defines more, and each has a precise role:

- **GET**: retrieve data. The browser sends this when you type a URL or click a link.
- **POST**: create a new resource. The server receives data it has never seen before.
- **PUT**: replace an existing resource. The server updates the record that matches the given ID.
- **DELETE**: remove a resource. The server deletes the record.

There are others (PATCH, HEAD, OPTIONS), but these four are all you need for a CRUD application. In the next chapter we will see how each method maps to a CRUD operation.

The URL is always the same (`this.API`). The server reads the method to decide what to do. This is **REST**: Representational State Transfer. The name is academic, but the idea is simple. Remember the constructor of our classes?

```
this.API = '/bibliotheca/public/api/publishers.php';
```

That URL is what REST calls a **resource**. Every time we call `fetch`, we point to that same URL:

```
response = await fetch(this.API, {
    method: 'POST',
    ...
});
```

The URL stays the same. Only the `method` changes: POST to create, PUT to update, DELETE to remove.

This is the key insight of the chapter: one URL, different methods. Everything else follows from here. You act on it by choosing the right HTTP method. The URL says *what*, the method says *what to do with it*. Instead of inventing separate URLs like `/api/publishers/create` and `/api/publishers/delete`, we use a single URL and let the HTTP method express the intent.

The separation between `load` and `render` is the same in every class: `load` talks to the server, `render` talks to the DOM. Once you understand `Publisher`, you understand them all. `Book` adds complexity (multiple authors, more fields), but the structure is the same.

Chapter 6

CRUD

6.1 The sign of the four

CRUD stands for Create, Read, Update, Delete. Every data-driven application does these four things. Nothing more, nothing less.

In the previous chapter we learned that HTTP has a method for each operation. Here is the complete mapping, from the browser to the database:

Operation	HTTP Method	Example
Create	`POST`	`INSERT` a new publisher
Read	`GET`	`SELECT` all publishers
Update	`PUT`	`UPDATE` a publisher's name
Delete	`DELETE`	`DELETE` a publisher

Three layers, one intention. The HTTP method arrives at the API file (e.g. `api/publishers.php`). In MVC architecture, this file is called the **controller**: it receives the request and decides what to do. It does not talk to the database directly. Instead, it calls the **model**: a PHP class (file) that lives in `src/` (outside `public/`, because it handles data, not HTTP requests). Each model — `Publisher.php`, `Category.php`, `Author.php`, `Book.php` — implements the CRUD methods internally: `insert`, `getAll`, `getById`, `update`, `delete`. None of them talks to the database directly either. They all use `DBMS.php`, a class we wrote that wraps PDO and exposes

simple methods: `fetchOne`, `fetchAll`, `execute`. Every model receives a `DBMS` instance in its constructor and uses it for every query. One class for database access, shared by all models. The controller calls the model, the model calls DBMS, DBMS talks to SQLite, and the controller returns the result as JSON.

6.2 Soft delete vs hard delete

Bibliotheca uses a `status` column (1 = active, 0 = disabled). When you disable a record, the row stays in the database — it just appears greyed out in the list. This is a **soft delete**: a PUT that sets `status` to 0.

When you delete a record, it is gone. `DELETE FROM` removes the row entirely. This is a **hard delete**: permanent, irreversible.

Why both? In a real application, you often want to keep the data. An author who is temporarily unavailable should not disappear from the database — their books still reference them. Disabling is safer. Deleting is for cleanup.

6.3 From form to database

In a classic website, clicking a button loads a new page. In Bibliotheca, JavaScript intercepts the action, calls `fetch` with the right HTTP method, and updates the page without reloading. Let us follow each operation step by step.

6.3.1 Create, POST

1. The user fills the form and clicks Save.
2. JavaScript builds the `payload` object from the form fields.
3. `fetch` sends a POST with the payload as JSON body.
4. The controller validates and calls the model's `insert`.
5. The model executes `INSERT INTO` via PDO.
6. The controller returns the new record as JSON.
7. JavaScript redirects to the list page.

6.3.2 Read, GET

1. The page loads and JavaScript calls `fetch` with GET.
2. The controller calls the model's `getAll` or `getById`.
3. The model queries via PDO and returns an array.
4. The controller encodes the array as JSON.
5. JavaScript receives the data and calls `render` to build the table rows or populate the form fields.

6.3.3 Update, PUT

1. The user clicks Edit — the browser goes to the form page with `?id=` in the URL.
2. `checkEdit` detects the ID and calls `load(id)`.
3. `load` fetches the record via GET, then `render` populates the form.

4. The user modifies the fields and clicks Save.
5. JavaScript builds the `payload` and adds the record ID.
6. `fetch` sends a PUT with the payload as JSON body.
7. The controller validates and calls the model's `update`.
8. JavaScript redirects to the list page.

6.3.4 Delete, DELETE

1. The user clicks Delete — a confirmation dialog appears.
2. JavaScript builds the `payload` with the record ID.
3. `fetch` sends a DELETE with the payload as JSON body.
4. The controller calls the model's `delete` (`DELETE FROM` — permanent).
5. JavaScript redirects to the list page.

Notice how every operation follows the same shape: build the payload, choose the method, call `fetch`, handle the response. The only things that change are the HTTP method and the data inside the payload.

Chapter 7

Validation

7.1 Three lines of defense

Validation is not optional. It is part of the craft.

1. **HTML** — we set constraints directly on the form fields: `required`, `maxlength`, `type="number"`, `min`, `max`. The browser enforces these before any code runs. We add `novalidate` to the `<form>` tag so the browser does not show its own popups — we want to control the error messages ourselves.
2. **JavaScript** — we delegate the interactive checks to the `validate()` method, which runs before sending the request. Inline error messages appear under each field, red borders highlight invalid inputs. This is for the user experience: fast feedback, no round-trip to the server.
3. **PHP** — we delegate the final check to the server. Never trust the client. The server validates everything again, because anyone can bypass JavaScript with a single line in the browser console.

Each layer validates as if it were the only one. HTML does not know JavaScript exists. JavaScript does not assume the server will catch what it missed. The server does not trust anything that came before it. They each see the data from their own point of view, and they each reject bad input independently. When you write the validation for one layer, forget the others exist. Do not think "HTML already checks this" or "the server will catch it anyway". Each layer must stand on its own and do its job to the best of its capabilities.

7.2 What we validate in our project

- **Required fields** — cannot be empty after trimming.
- **String length** — names capped at 100 characters, titles at 255.
- **Numeric ranges** — pages between 1 and 99999, published year from 1000 to the current year.
- **Date format and range** — author birthdate must be a valid `YYYY-MM-DD`, year >= 1000, not in the future.
- **Duplicates** — server-side check with the model's `exists()` method. Returns HTTP 409 (Conflict) if the record already exists.
- **Dependencies** — returns HTTP 422 (Unprocessable Entity) if the record has linked books and cannot be deleted or disabled.
- **Input filtering** — numeric fields only accept digit keys.

7.3 Input normalization

Before data reaches the database, the server normalizes it. This is not cosmetic — it is data entry control. If you let "eINAUDI", " Einaudi ", and"einaudi" into the database, you have three records for the same publisher. Same with numbers: if a field expects an integer (like pages or published year), then only digits should be allowed. Not letters, not decimals, not special characters. The type of number matters: an integer is not a float, a year is not a price. Look at the column type in the database table: if it says `INTEGER`, the input field should only accept digits. A winning strategy: define what you expect based on where the data will be stored, and reject everything else at the point of entry.

- `trim()` — no leading or trailing spaces.
- `ucwords(strtolower())` — "eINAUDI" becomes "Einaudi".

The user types whatever they want. The server stores it in a consistent format, every time.

7.4 Sanitization

Formatting makes input look right. Sanitization makes it *safe*.

Every string that enters the API goes through three filters before anything else happens:

```
$name = mb_substr(strip_tags(trim($data['name'] ?? '')), 0, 100);
```

- **`trim()`** — removes leading and trailing whitespace.
- **`strip_tags()`** — removes any HTML or PHP tags. A name like `<script>alert('xss')</script>` becomes `alert('xss')`. The dangerous part is gone before the value reaches the database.
- **`mb_substr()`** — enforces a maximum length. Even if someone sends a megabyte of text, only the first 100 characters survive.

The order matters: trim first, strip second, truncate last. And this happens on the server — never trust the client's `maxlength` attribute alone.

7.5 Error display: how do we tell the user something is wrong?

No `alert()`. We use inline messages:

```
<span class="error" id="publisher-name-error"></span>
```

Each input field has a matching `<span>` for its error message. The JavaScript class has two methods for this:

```
showError(fieldId, message) {
    const input = document.querySelector('#' + fieldId);
    const error = document.querySelector(
        '#' + fieldId + '-error'
    );
    input.classList.add('invalid');
    error.textContent = message;
}
```

```
clearErrors() {
    const errors = document.querySelectorAll('.error');
    for (let i = 0; i < errors.length; i++) {
        errors[i].textContent = '';
    }
    const invalids = document.querySelectorAll('.invalid');
    for (let i = 0; i < invalids.length; i++) {
        invalids[i].classList.remove('invalid');
    }
}
```

`showError` marks one field as invalid and shows the message. `clearErrors` resets all fields before a new validation pass. The `validate()` method calls `clearErrors` first, then checks each field and calls `showError` for every problem it finds.

7.6 Server errors

When the server rejects the input (409 for duplicates, 422 for dependencies, 400 for bad data), JavaScript reads the response and shows the message inline — same place, same pattern:

```
if (!response.ok) {
    const result = await response.json();
    this.showError('publisher-name', result.error);
    return;
}
```

Whether the error comes from client-side validation or from the server, the user sees it in the same way.

7.7 Exceptions: who catches what

Validation handles *expected* problems — empty fields, duplicates. But what about *unexpected* failures? The database file is corrupted. The disk is full. A table was dropped. These are exceptions.

In PHP, PDO is configured with `ERRMODE_EXCEPTION`: when something goes wrong, it throws a `PDOException`. The question is: who catches it?

Not DBMS. The database wrapper does not know *how* to handle the error. Should it return null? An empty array? Log something? It depends on who is calling. A web API needs a JSON response. A CLI script needs a console message. A test needs an assertion failure. DBMS cannot know any of this, so it lets the exception rise.

Not the Model. Publisher, Book, Author — they have the same problem. They do not know the context. They pass the exception upward.

The controller catches it. The API file is the only place that knows it must respond with JSON and an HTTP status code:

```
try {
    $db = new DBMS(
        __DIR__ . '/../../sql/bibliotheca.db'
```

```
    );
    $publisher = new Publisher($db);
    // ... all the logic ...
} catch (\Exception $e) {
    http_response_code(500);
    echo json_encode(['error' => 'Internal server error']);
}
```

JavaScript catches too. Network failures (server down, timeout) throw exceptions at the `fetch` level:

```
try {
    const response = await fetch(this.API);
    if (!response.ok) {
        throw new Error(response.statusText);
    }
    const data = await response.json();
    this.render(data);
} catch (error) {
    alert('Unable to load data.');
}
```

The principle: **catch where you can act.** The lower layers report the problem (by throwing). The upper layers handle it (by catching). An exception is a signal that travels upward — like pain traveling from an organ to the brain. The kidney does not decide how to inform the patient. The brain does.

Chapter 8

Permissions

8.1 The problem

Your code works from the terminal. You open the browser and get a blank page, or worse, a 500 Internal Server Error. The reason is almost always permissions.

Apache runs as a user called `www-data`. Your files belong to your user (e.g., `bc`). If `www-data` cannot read a file, the page is blank. If it cannot write, the database operations fail silently.

8.2 How Linux permissions work

Every file has three sets of permissions: owner, group, others.

```
-rw-r--r-- 1 youruser youruser 28672 bibliotheca.db
```

- `rw-` — owner (`youruser`) can read and write.
- `r--` — group (`youruser`) can only read.
- `r--` — others can only read.

Apache (`www-data`) is "others" here — it can read but not write. That is why SELECT works but INSERT fails.

8.3 The fix

Give the group `www-data` write access to the database file and its directory (SQLite needs the directory for its journal file):

```
sudo chgrp www-data /var/www/html/bibliotheca/sql/bibliotheca.db
sudo chmod 664 /var/www/html/bibliotheca/sql/bibliotheca.db
sudo chgrp www-data /var/www/html/bibliotheca/sql/
sudo chmod 775 /var/www/html/bibliotheca/sql/
```

- `664` — owner reads/writes, group reads/writes, others read.
- `775` — same for directories, plus execute (needed to list contents).

Check the result with `ls -la`:

```
ls -la sql/
```

The terminal is your best friend for diagnosing permission problems. Always verify with `ls -la` after changing permissions. Get comfortable with the command line — it should always be your first choice. A production server has no graphical interface. There is no file manager, no right-click, no properties dialog. There is only the terminal. The sooner you make it your natural environment, the better.

8.4 Every time you recreate the database

When you delete and recreate `bibliotheca.db`, the new file belongs to your user with default permissions. You must set them again:

```
rm bibliotheca.db
sqlite3 bibliotheca.db ".read ddl/create_table.sql"
sqlite3 bibliotheca.db ".read dml/insert.sql"
sudo chgrp www-data bibliotheca.db
sudo chmod 664 bibliotheca.db
```

This is easy to forget. And when you forget, the application reads data fine but fails on any write operation with:

```
SQLSTATE[HY000]: General error: 8 attempt to write a readonly database
```

If you see this error, check the permissions first. Nine times out of ten, that is the problem.

8.5 Why `public/` matters

Notice the project structure: the `src/` folder (models, DBMS) and the `sql/` folder (database) are outside `public/`. Only the files inside `public/` are reachable from the browser.

This is a permission boundary too. Even if someone guesses the path to `sql/bibliotheca.db`, Apache will not serve it — the `.htaccess` rewrite rule sends everything through `index.php`, and the router only accepts whitelisted routes.

The file system layout is your first line of defense. Keep sensitive files outside the web root.

Chapter 9

Debugging

9.1 The blank page

The most frustrating thing in web development: your code looks correct, but the browser shows nothing. No error, no message, just white. This means PHP crashed before producing any output.

The browser cannot help you here. You need to look elsewhere.

9.2 Think in layers

When something breaks, remember the architecture from Chapter 06: JavaScript calls the controller, the controller calls the model, the model calls DBMS, DBMS talks to SQLite. The bug lives in one of these layers. Your job is to find which one.

Start from the outside and work inward:

1. Is the browser showing a JavaScript error? **Console**
2. Is the server returning an error code? **Network tab**
3. Is PHP crashing? **Apache error log**
4. Is the SQL wrong? **SQLite command line**

Be methodical. Narrow it down.

9.3 Apache error log

The server-side truth lives here:

```
sudo tail -20 /var/log/apache2/error.log
```

This log captures every PHP fatal error, every uncaught exception, every permission problem. When the browser shows a 500, this log tells you why.

For real-time monitoring, use `-f` — the log updates as requests come in:

```
sudo tail -f /var/log/apache2/error.log
```

Keep this running in a terminal while you test in the browser. You will see errors the moment they happen.

Common errors you will find:

- `could not find driver` — a PHP extension is missing.
- `attempt to write a readonly database` — permissions (Chapter 08).
- `Uncaught PDOException` — a SQL error, with the exact line number.

9.4 curl: testing the API

Before blaming the frontend, test the backend directly:

```
curl http://localhost/bibliotheca/public/api/publishers.php
```

If this returns JSON, the backend works. If it returns nothing or an error, the problem is server-side.

For POST requests, remember that the controller requires a CSRF token (Chapter 10). Without it, you will get a 403. For a quick diagnostic, you can test a GET first to confirm the API is alive, then check the Network tab for POST issues.

Use `-v` for verbose output — it shows HTTP status codes, headers, and the full request/response cycle:

```
curl -v http://localhost/bibliotheca/public/api/publishers.php
```

The `-v` flag is one of the most useful debugging tools you have. It shows you exactly what the browser and server are saying to each other.

9.5 Browser developer tools

Press F12. Three tabs matter:

- **Console** — JavaScript errors appear here. If `fetch` fails or your code has a syntax error, this is where you see it.
- **Network** — every HTTP request the browser makes. Click on a request to see the status code, request body, and response body. If a fetch returns 500, click on it to see the server's error message.
- **Elements** — the live DOM. Useful to check if JavaScript actually built the HTML you expected. Remember the DOM diagram from Chapter 05: what you see here is the tree, not the source file.

9.6 PHP debugging tools

When you need to inspect a value inside your PHP code, two tools:

`error_log()` — writes to the Apache error log without breaking the response:

```
error_log('Publisher name: ' . $name);
error_log(print_r($data, true));
```

Check the output with `tail -f /var/log/apache2/error.log`. This is safe to use in API controllers — it does not corrupt the JSON response.

var_dump() — prints the value directly to the output:

```
var_dump($data);
exit;
```

Useful for quick inspection, but it breaks the JSON response. Use it only for temporary debugging, never in production code. Remove it when you are done.

9.7 PHP from the command line

You can run a PHP file directly to see errors without Apache:

```
php /var/www/html/bibliotheca/public/api/publishers.php
```

This bypasses Apache and permissions. If it works here but not in the browser, the problem is Apache configuration or permissions (Chapter 08).

9.8 SQLite from the command line

You can inspect the database directly:

```
sqlite3 sql/bibliotheca.db
```

Useful commands inside the SQLite shell:

```
-- List all tables
.tables

-- Show the schema of a table
PRAGMA table_info(book);

-- Show all indexes in the database
.indices

-- Show the indexes of a specific table
PRAGMA index_list(book);

-- Show which columns an index covers
PRAGMA index_info(idx_book_publisher);

-- Check if foreign keys are enforced
PRAGMA foreign_keys;

-- Show the CREATE statement for a table
.schema book

-- Run a quick query
SELECT * FROM publisher ORDER BY name;
```

`PRAGMA table_info` is especially useful: it shows the column name, type, whether it allows NULL, its default value, and whether it is part of the primary key. Learn to use it — it is faster than reading the schema file.

9.9 The golden rule

When something breaks, resist the urge to change random things hoping the problem goes away. Read the error message. Follow the chain: browser console network tab Apache log PHP command line SQLite shell. The answer is always in one of these places. Your job is not to fix — it is to *find*. Once you find it, the fix is usually obvious.

Chapter 10

Security

10.1 What we already have

Previous chapters covered the essentials:

- **SQL injection** — prepared statements with named parameters (Chapter 04).
- **XSS** — `htmlspecialchars()` in PHP, `textContent` in JavaScript (Chapter 07).
- **Input validation** — server-side checks on every field (Chapter 07).
- **File permissions** — keeping sensitive files outside the web root (Chapter 08).

These are non-negotiable. They are part of the code you have already written. This chapter adds the layers that protect *around* your code.

10.2 CSRF — protecting write operations

In Chapter 05 we saw that JavaScript sends an `X-CSRF-Token` header with every POST, PUT, and DELETE request. But what is CSRF, and why do we need that token?

10.2.1 The attack

Imagine you have Bibliotheca open in your browser. A malicious website in another tab contains hidden JavaScript that sends a POST request to your `api/publishers.php` with `{"name": "HACKED"}`. The browser sends it — from its perspective, it is just another HTTP request to localhost, and it attaches your session cookie automatically.

This is **CSRF** (Cross-Site Request Forgery): a malicious site tricks the browser into making requests to *your* application, using *your* session.

10.2.2 The defense: a secret token

The solution is a **CSRF token**: a long random string that only our pages know. The malicious site cannot read it because the browser's **Same-Origin Policy** prevents one site from reading another site's content.

The flow:

1. When PHP renders the page (`index.php`), it generates a random token and stores it in the session:

```
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

2. The token is injected into the HTML as a `<meta>` tag:

```
<meta name="csrf-token" content="a7f3b9c2e1d4...">
```

3. JavaScript reads the token and includes it in every POST, PUT, and DELETE request — the same header we saw in Chapter 05:

```
headers: {
    'Content-Type': 'application/json',
    'X-CSRF-Token': document.querySelector(
        'meta[name="csrf-token"]'
    ).content
}
```

4. The controller (the API file) verifies the token before processing the request:

```
Csrf::verify();  // 403 if token is missing or wrong
```

The malicious site cannot read the `<meta>` tag from our page (Same-Origin Policy), so it cannot send the correct token. The controller rejects its request with `403 Forbidden`.

10.2.3 The Csrf class

The `Csrf` class (`src/Csrf.php`) has three methods:

- **`Csrf::start()`** — starts a PHP session (with secure cookie settings).
- **`Csrf::token()`** — returns the token, generating one if needed.
- **`Csrf::verify()`** — compares the `X-CSRF-Token` header against the session; exits with 403 on mismatch.

GET requests do not need CSRF protection — they only read data, they never modify it. Only POST, PUT, and DELETE are checked.

10.2.4 Verify it works from the terminal

```
# Without token: 403
curl -s -o /dev/null -w "%{http_code}" \
     -X POST \
     http://localhost/bibliotheca/public/api/publishers.php \
     -H "Content-Type: application/json" \
     -d '{"name": "Test"}'
```

```
# With token: 200
TOKEN=$(curl -s -c /tmp/c.txt \
    http://localhost/bibliotheca/public/ \
    | grep csrf-token \
    | sed 's/.*content="\([^"]*\)".*/\1/')
```

```
curl -s -o /dev/null -w "%{http_code}" \
     -b /tmp/c.txt -X POST \
     http://localhost/bibliotheca/public/api/publishers.php \
     -H "Content-Type: application/json" \
     -H "X-CSRF-Token: $TOKEN" \
     -d '{"name": "Test"}'
```

The first request has no token — the server rejects it. The second extracts the token from the page and sends it — the server accepts it. Try both from your terminal.

10.3 Sessions and session fixation

HTTP is stateless: each request is independent, the server does not remember who you are. **Sessions** solve this. When you visit the application, PHP creates a session — a small file on the server identified by a random ID. The browser stores this ID in a cookie and sends it with every request. That is how the server knows you are still you.

When you log in, PHP writes your user ID into the session. From that point, every request carries your identity.

A **session fixation** attack exploits this: the attacker gives you a session ID they already know (via a crafted URL or a cookie), then waits for you to log in. Now they share your authenticated session.

The defense is simple: regenerate the session ID after any privilege change.

```php
session_start();
session_regenerate_id(true);  // true = delete the old session file
```

The `true` parameter is important. Without it, the old session file survives and the attacker's ID still works.

Bibliotheca calls `session_regenerate_id(true)` in `Auth::login()` immediately after verifying credentials. Our `Csrf::start()` calls `session_start()` with secure cookie settings:

```php
session_start([
    'cookie_httponly' => true,
    'cookie_samesite' => 'Strict',
]);
```

- `cookie_httponly` — JavaScript cannot read the session cookie. This blocks XSS attacks that try to steal session IDs.
- `cookie_samesite` — the browser will not send the cookie with requests from other sites. This is a second layer of CSRF defense.

10.4 Security headers

HTTP headers tell the browser how to behave. Without them, the browser uses permissive defaults that attackers can exploit.

Add these headers in your front controller or in Apache configuration:

```
// Prevent MIME-type sniffing  browser trusts the Content-Type header
header('X-Content-Type-Options: nosniff');

// Deny embedding in iframes  blocks clickjacking attacks
header('X-Frame-Options: DENY');

// Only send the origin as referrer, not the full URL with parameters
header('Referrer-Policy: strict-origin-when-cross-origin');

// Control what browser features the page can use
header('Permissions-Policy: camera=(), microphone=(), geolocation=()');
```

10.4.1 Content-Security-Policy

CSP is the most powerful security header. It tells the browser exactly which resources are allowed to load:

```
header("Content-Security-Policy: default-src 'self'; "
     . "style-src 'self' https://cdnjs.cloudflare.com; "
     . "font-src https://cdnjs.cloudflare.com; "
     . "script-src 'self'");
```

What this means:

- `default-src 'self'` — by default, only load resources from our own domain.
- `style-src 'self' https://cdnjs.cloudflare.com` — CSS from us and from the Font Awesome CDN.
- `font-src https://cdnjs.cloudflare.com` — fonts only from Font Awesome.
- `script-src 'self'` — JavaScript only from our own domain. No inline scripts, no external scripts. If an attacker injects a `<script>` tag, the browser refuses to execute it.

CSP does not replace `htmlspecialchars()`. It is a safety net — if your escaping has a gap, CSP catches the fall.

10.5 Rate limiting

Without rate limiting, anyone can hammer your API with thousands of requests per second. This can exhaust your server or brute-force data through your endpoints.

A simple approach for a small application: count requests per IP in the session.

```
function checkRateLimit(int $maxRequests = 60, int $windowSeconds = 60): void
{
    $now = time();
    $key = 'rate_limit';

    if (!isset($_SESSION[$key])) {
        $_SESSION[$key] = ['count' => 0, 'start' => $now];
    }

    if ($now - $_SESSION[$key]['start'] > $windowSeconds) {
```

```
        $_SESSION[$key] = ['count' => 0, 'start' => $now];
    }

    $_SESSION[$key]['count']++;

    if ($_SESSION[$key]['count'] > $maxRequests) {
        http_response_code(429);
        header('Retry-After: ' . $windowSeconds);
        echo json_encode(['error' => 'Too many requests']);
        exit;
    }
}
```

HTTP 429 (Too Many Requests) is the correct status code. The `Retry-After` header tells the client how long to wait.

For production systems, rate limiting belongs in Apache (`mod_ratelimit`), Nginx, or a reverse proxy — not in PHP. But understanding the mechanism at the application level is what matters for learning.

10.6 HTTPS

Everything above is pointless if the connection is not encrypted. Without HTTPS:

- Anyone on the network can read your session cookies.
- Anyone can modify the HTML in transit and inject scripts.
- Security headers can be stripped by a man-in-the-middle.

In development on localhost, HTTP is fine. In production, HTTPS is mandatory. There are no exceptions.

10.6.1 How it works

HTTPS uses TLS (Transport Layer Security) to encrypt the connection. The server holds a **certificate** — a file that proves its identity. When the browser connects, the server presents the certificate, they negotiate encryption keys, and from that point every byte is encrypted.

The certificate contains the domain name, the public key, the issuer, and an expiry date. It is signed by a **Certificate Authority** (CA) that the browser trusts. If the signature does not match or the certificate is expired, the browser shows a warning.

10.6.2 Where it lives

On a Debian/Apache server, certificates typically live in `/etc/letsencrypt/live/yourdomain/`. Two files matter:

- `fullchain.pem` — the certificate (public)
- `privkey.pem` — the private key (never share this)

Apache references them in the virtual host configuration:

```
SSLCertificateFile /etc/letsencrypt/live/yourdomain/fullchain.pem
SSLCertificateKeyFile /etc/letsencrypt/live/yourdomain/privkey.pem
```

10.6.3 Getting a certificate

With Let's Encrypt and `certbot`, it takes two commands:

```
sudo apt install certbot python3-certbot-apache
sudo certbot --apache
```

`certbot` obtains the certificate, configures Apache, and sets up automatic renewal. The certificate expires every 90 days, but `certbot` renews it automatically via a systemd timer.

10.7 The layers

Security is not one thing. It is layers, each catching what the previous one missed:

- **Prepared statements** SQL injection
- **Output escaping** XSS
- **CSRF tokens** cross-site request forgery
- **Input validation** bad data, out-of-range values
- **Session management** session fixation, session theft
- **Security headers** clickjacking, MIME sniffing, CSP bypass
- **Rate limiting** brute force, denial of service
- **HTTPS** eavesdropping, tampering

No single layer is sufficient. Together, they make an application that is genuinely hard to attack.

10.8 The rule

Security is not a feature you add at the end. It is a property of every line of code you write, every header you set, every default you accept or override.

Remember the principle from Chapter 07: each layer validates as if it were the only one. The same applies here. Prepared statements do not assume CSP will block the injection. CSRF tokens do not assume HTTPS will stop the attacker. Each defense stands on its own.

The attacker only needs one gap. You need them all closed.

Appendix A

How It Works

A.1 Request and response

Everything on the web is built on one mechanism: a client sends a **request**, a server sends back a **response**. That is it. Every click, every page load, every form submission, every API call — request, response. HTML, CSS, JavaScript, frameworks, databases — they all exist to produce or consume requests and responses. If you understand this, you understand the web.

In our application, every request that JavaScript makes to the server goes through one function: `fetch`. It is the bridge between the browser and the server. Without it, JavaScript can change the page but cannot talk to anyone. This is why learning JavaScript matters. It can change the color of a button or make elements appear and disappear, but it does much more than that. It is the nervous system of the application (Chapter 00): the language that ferries data between the browser and the server. `fetch` is its Charon.

A.2 Open your terminal

The best way to see this in action is to watch it from the outside. Not with the browser — the browser does too much at once. Use `curl`, which does exactly one thing: send a request, show the response.

A.3 The first request

Click "Categories" on the menu. What actually happens? Let us ask `curl`:

```
curl -v http://localhost/bibliotheca/public/categories \
    2>&1 | grep "^>"
```

```
> GET /bibliotheca/public/categories HTTP/1.1
> Host: localhost
> User-Agent: curl/7.88.1
> Accept: */*
>
```

This is the **request**. The browser sends these exact bytes to Apache on port 80 (or 443 for HTTPS). The first line says: send me the page at this URL, using HTTP/1.1. (It could be a page, an image, a JSON response — the browser does not know in advance. It just asks.) The rest are headers — metadata about the request.

Now the response:

```
curl -s http://localhost/bibliotheca/public/categories
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="csrf-token" content="a7f3b9...">
    <meta name="authenticated" content="0">
    <title>Bibliotheca</title>
    <link rel="stylesheet" href="...style.css">
</head>
<body>
    <header>...</header>
    <main>
        <h2>Categories</h2>
        <table id="category-table">
            <thead>
                <tr><th>Name</th><th></th></tr>
            </thead>
            <tbody></tbody>
        </table>
    </main>
    <script src="...categories.js"></script>
</body>
</html>
```

Look at the `<tbody>`. It is empty. There are no records.

A.4 Where are the records?

They are not in this response. The HTML contains the structure — the skeleton — but no data. The table is an empty container waiting to be filled.

The records live in the database. The only way to get them is to call the API:

```
curl -s http://localhost/bibliotheca/public/api/categories.php
```

```
[
  {"category_id":1,"name":"Fiction","status":1},
  {"category_id":2,"name":"Science","status":1},
  {"category_id":3,"name":"History","status":0}
]
```

There they are. JSON, not HTML. Data, not structure.

A.5 Two requests, one page

This is the key: **loading a page takes two requests**.

1. **First request** — the browser asks for the page. Apache rewrites the URL through `.htaccess`, the router loads the template, PHP sends back HTML. The table is empty.

2. **Second request** — JavaScript starts (the `<script>` tag at the bottom of the page), calls `fetch` to the API, gets JSON back, and fills the table with `createElement` and `textContent`.

When you use the browser, this happens automatically — so fast you never notice the table is empty for a fraction of a second. With `curl`, you see the truth: `curl` does not execute JavaScript, so the second request never happens. The table stays empty.

You can see both requests in the browser's Network tab (F12). The first returns HTML. The second returns JSON. Two trips, one page.

A.6 What travels on the wire

When the browser sends a request, it is not a file. It is a block of text transmitted over a TCP connection to port 80 (or 443 for HTTPS).

```
> GET /bibliotheca/public/categories HTTP/1.1
> Host: localhost
> Accept: */*
>
```

The first line is the **request line**: method, path, protocol. The lines that follow are **headers**: key-value pairs. The empty line marks the end. A GET has no body; a POST carries one after the blank line.

The server responds in the same format:

```
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=UTF-8
< Set-Cookie: PHPSESSID=...; HttpOnly; SameSite=Strict
<
<!DOCTYPE html>...
```

Status line, headers, blank line, body. That is all HTTP is: structured text over a wire.

No files are written to disk. Apache has a process (managed by its **MPM**, Multi-Processing Module) listening on a TCP socket. The bytes arrive in memory, Apache interprets them, calls PHP, and sends the response back on the same connection.

With HTTP/1.1, the connection stays open briefly (**keep-alive**) so the browser can reuse it for the CSS, the JavaScript, and the favicon without opening a new connection each time.

A.7 What Apache does

Apache receives the request and reads the path. Since `/categories` is not a real file, `.htaccess` rewrites it:

```
RewriteRule ^(.+)$ index.php?route=$1 [QSA,L]
```

Now Apache calls PHP with `index.php?route=categories`. The router checks the whitelist, finds "categories", and loads `pages/categories.php` inside the HTML shell (header, nav, main, footer). PHP sends the complete HTML back to Apache, Apache sends it to the browser.

A.8 What the browser does

The browser receives the HTML and builds the **DOM** (Document Object Model) — an in-memory tree of objects:

```
table #category-table
 thead
    tr
        th  "Name"
 tbody  (empty)
```

Every tag becomes a node. JavaScript works on this tree, not on the HTML text. When it calls `createElement('tr')` and `appendChild(row)`, it adds nodes to the tree and the browser redraws the screen.

The `<script>` tag at the bottom triggers the second request. It sits at the bottom so the DOM is already built when JavaScript starts — otherwise `querySelector` would not find the elements.

A.9 What JavaScript does

The browser loads `js/categories.js` (the `<script>` tag at the bottom of the HTML). This file creates a `Categories` object. The constructor calls `load()`. These are methods we wrote — `load` and `render` are not part of JavaScript, they are our convention (Chapter 05). The only built-in part is `fetch`, which sends the HTTP request. `this.API` is a property set in the constructor — it holds the URL of the API endpoint:

```
this.API = '/bibliotheca/public/api/categories.php';
```

So when we write:

```
async load() {
    const response = await fetch(this.API);
    const categories = await response.json();
    this.render(categories);
}
```

`fetch(this.API)` sends a GET request to that URL — the same request we just made with `curl`. Notice there is no `method: 'GET'` here: when you call `fetch` with just a URL, it defaults to GET. Writing:

`fetch(this.API)`

and:

`fetch(this.API, { method: 'GET' })`

is the same thing. You only need to specify the method for POST, PUT, and DELETE.

The server responds through the API endpoint (`api/categories.php`), which calls the model (`src/Category.php`), which calls DBMS (`src/DBMS.php`), which queries SQLite (`sql/bibliotheca.db`), and if everything went as it should, the server responds with JSON — a list of records if we sent GET, the new or updated record if we sent POST or PUT, a confirmation if we sent DELETE.

Now you see why the directories are organized that way. `src/` and `sql/` are outside `public/` because they are private — only PHP can reach them. `api/` and `pages/` are inside `public/` because they need to receive HTTP requests. The directory structure is not arbitrary. It reflects who can talk to who.

One API file and one model per entity. Every operation on categories — read, create, update, delete — goes through those same two files. The pages and JavaScript files instead are two: one for the list (`categories.php` + `categories.js`) and one for the form (`category.php` + `category.js`).

In the case of a GET, back in the browser, `load` receives the JSON data. `render` builds the DOM: for each category, it creates a `<tr>`, sets the text with `textContent`, and appends it to the `<tbody>`. The empty table fills up. For POST, PUT, and DELETE the flow is different — we will see that shortly in "What happens when you click Save".

A.10 How data changes shape

A single piece of data passes through four representations on its way from disk to screen:

Layer	Form
SQLite	Row in a table
PHP	Associative array
Network	JSON text
JavaScript	Object with properties

```
SQLite:  | 1 | Fiction | 1 |

PHP:     ['category_id'=>1, 'name'=>'Fiction',
          'status'=>1]

JSON:    {"category_id":1,"name":"Fiction",
          "status":1}

JS:      {category_id: 1, name: "Fiction",
          status: 1}
```

Each layer only knows about the one directly below it. JavaScript has no idea SQLite exists; it sees a URL that returns JSON. The model has no idea JavaScript exists; it returns an array to the controller.

A.11 The chain

The full chain, with the actual files. Notice the two requests: the first (steps 1-3) brings the HTML, the second (steps 4-8) brings the data.

```
First request  the page:

  Browser
```

```
  1  .htaccess              (rewrites URL)
  2  index.php              (router, loads template)
  3  pages/categories.php   (HTML skeleton)
       HTML response (empty table + script tag)

Second request  the data:

  4  js/categories.js       (fetch)
  5  api/categories.php     (controller)
  6  src/Category.php       (model)
  7  src/DBMS.php            (database wrapper)
  8  sql/bibliotheca.db      (SQLite)
       JSON response (the records)
```

And back: SQLite returns rows `DBMS.php` returns arrays the model (`Category.php`) returns to the controller (`api/categories.php`) the controller encodes JSON JavaScript (`categories.js`) receives it `render` builds the DOM the user sees the table.

This is MVC in action: the **Model** (`src/Category.php`) handles the data, the **Controller** (`api/categories.php`) handles the request, the **View** (`pages/categories.php` + `js/categories.js`) handles what the user sees.

Take note of this mapping. Learn which directory holds what. Even if it all feels unclear right now, this map will be the thing that makes everything click.

A.12 What happens when you click Save

The list page reads. The form page writes. What happens when the user fills a form and clicks Save?

```
# JavaScript sends this (you can simulate with curl):
curl -X POST \
     -H "Content-Type: application/json" \
     -H "X-CSRF-Token: a7f3b9..." \
     -d '{"name":"Biography"}' \
     http://localhost/bibliotheca/public/api/categories.php
```

The same URL as the GET. The only difference is the method: POST instead of GET. When JavaScript writes:

```
response = await fetch(this.API, {
    method: 'POST',
    ...
});
```

that `'POST'` becomes the first word of the HTTP request line: `POST /api/categories.php HTTP/1.1`. It travels over the wire like any other text. On the server side, PHP reads it from `$_SERVER['REQUEST_METHOD']`. The controller uses this value to decide what to do:

- **POST** validate, `insert`, return the new record
- **PUT** validate, `update`, return the updated record
- **DELETE** check dependencies, `delete`, confirm

A.13 What `fetch` sends to the controller (`api/categories.php`) and what it gets back

Look at a complete `fetch` call. Remember, `this.API` is the URL we defined in the constructor (`/bibliotheca/public/api/categories.php`):

The `payload` is the JavaScript object that holds the data we want to send — built from the form fields:

```
const payload = { name: 'Biography' };
```

For a category, the payload is simple — just a name. For a book, it carries more fields — as many as the database table columns we need to insert or update:

```
const payload = {
    title: 'The Art of War',
    publisher_id: 1,
    category_id: 3,
    pages: 128,
    published: 500,
    author_ids: [5]
};
```

The structure changes, the mechanism does not. Then `fetch` sends it:

```
response = await fetch(this.API, {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'X-CSRF-Token': document.querySelector(
            'meta[name="csrf-token"]'
        ).content
    },
    body: JSON.stringify(payload)
});
```

Everything inside this call becomes part of the HTTP request that travels to the server:

- **`method: 'POST'`** PHP reads it from `$_SERVER['REQUEST_METHOD']`.
- **`'Content-Type': 'application/json'`** tells PHP that the body is JSON. PHP reads the body with `json_decode(file_get_contents('php://input'))`.
- **`'X-CSRF-Token': ...`** the security token. PHP reads it from the headers and compares it to the session. If it does not match, the request is rejected with 403.
- **`body: JSON.stringify(payload)`** the actual data. The `payload` object is converted to a JSON string because HTTP only carries text.

Nothing is hidden. Everything that `fetch` sends, PHP can read. The request is just structured text — method, headers, body — and each part has a purpose.

A.14 Inside the controller (`api/categories.php`)

The controller runs on the server. You never see it work — there is no visible output, no window, no log on screen. Out of sight, out of mind. But it is the most important file in the chain.

What does `api/categories.php` actually do when a request arrives? The first thing it does is load the model and the database wrapper, and create the objects it needs:

```
require_once __DIR__ . '/../../src/DBMS.php';
require_once __DIR__ . '/../../src/Category.php';

$db = new DBMS(__DIR__ . '/../../sql/bibliotheca.db');
$category = new Category($db);
```

Notice the path: `../../src/` — the controller reaches outside `public/` to get to the private files. The model receives the DBMS instance in its constructor, so it can query the database.

Then it reads the HTTP method and decides what to do:

```
$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'GET') {
    // read: call the model, return JSON
} elseif ($method === 'POST') {
    // create: read the body, validate, insert
} elseif ($method === 'PUT') {
    // update: read the body, validate, update
} elseif ($method === 'DELETE') {
    // delete: read the body, check dependencies, delete
}
```

One file, one `if/elseif` chain. The HTTP method decides the branch. This is why REST works (Chapter 05): the same URL handles all four operations, and the method tells the controller which one.

For POST and PUT, the controller reads the body that `fetch` sent:

```
$data = json_decode(
    file_get_contents('php://input'), true
);
$name = trim(strip_tags($data['name'] ?? ''));
```

`php://input` is where PHP finds the request body — the JSON string that `JSON.stringify(payload)` created on the JavaScript side. `json_decode` turns it back into a PHP array. Then the controller validates, normalizes, and calls the model:

```
if ($name === '') {
    http_response_code(400);
    echo json_encode(['error' => 'Name is required']);
    exit;
}

$id = $category->insert($name);
echo json_encode(['category_id' => $id, 'name' => $name]);
```

If something is wrong, the controller sets an error status code and stops. If everything is fine, it calls the model and returns the result as JSON. The controller never touches the database directly — it delegates to the model, and the model delegates to DBMS.

A.15 Below the surface, under the browser's surfing: src/

When the controller (`api/categories.php`) calls `$category->insert($name)`, what happens? Open `src/Category.php`:

```
public function insert(string $name): int
{
    $sql = "INSERT INTO category (name) VALUES (:name)";

    return $this->db->insert($sql, [':name' => $name]);
}
```

The model knows the SQL — it knows the table name, the column names, the query structure. But it does not execute the query itself. It passes the SQL and the parameters to `$this->db`, which is the DBMS instance it received in its constructor.

Now open `src/DBMS.php`. Before executing a query, DBMS binds each parameter with the correct type:

```
private function bindParams(\PDOStatement $stmt,
                            array $params): void
{
    foreach ($params as $key => $value) {
        $type = match (true) {
            is_int($value)  => PDO::PARAM_INT,
            is_bool($value) => PDO::PARAM_BOOL,
            is_null($value) => PDO::PARAM_NULL,
            default         => PDO::PARAM_STR,
        };

        $stmt->bindValue($key, $value, $type);
    }
}
```

An integer is bound as `PARAM_INT`, a null as `PARAM_NULL`, a string as `PARAM_STR`. The database receives the right type, not just text.

We wrote this function (that in a class is more correctly named a method, just as variables are named properties). It is not part of PHP or PDO — it is ours. Why? Because we control the type of data at every level (Chapter 07). JavaScript validates the input, PHP normalizes it, and here — at the deepest point — we make sure the database receives an integer as an integer, not as a string that looks like a number. Instead of repeating this logic in every function, we write it once and reuse it everywhere. This is what it means to have dominion over the code: we make PDO work for us, not the other way around.

Then the insert function uses it:

```
public function insert(string $sql,
                       array $params = []): int
{
    $stmt = $this->pdo->prepare($sql);
    $this->bindParams($stmt, $params);
    $stmt->execute();
    return (int) $this->pdo->lastInsertId();
}
```

DBMS uses PDO — PHP's built-in database layer. `prepare` creates the query with placeholders (`:name`). `bindParams` attaches each value with its type. `execute` runs the query. `lastInsertId` returns the ID of the new row.

This is the deepest point of the chain. From here, the data travels back up: DBMS returns the ID to the model, the model returns it to the controller, the controller wraps it in JSON and sends it back to `fetch`.

Three files, three responsibilities:

File	Role	Handles
`api/categories.php`	Controller	HTTP
`src/Category.php`	Model	SQL
`src/DBMS.php`	Wrapper	PDO

Each one does one thing.

A.16 What comes back

What the controller sends back depends on what you asked — which HTTP method you used:

- **GET** the server returns data as JSON: a list of records, or a single record. This is the only method that returns data to display.
- **POST, PUT, DELETE** the server does not return data to display. It returns a confirmation (`{"category_id":4,"name":"Biography"}`) or an error (`{"error":"Category already exists"}`).

Every response also carries a **status code**: 200 (OK), 400 (bad input), 401 (not logged in), 409 (duplicate), 422 (dependency error).

The user never sees this JSON directly. Unlike a traditional website, the server does not redirect to a new page. It just sends JSON back. It is JavaScript that reads the response and decides what to do: redirect to the list on success, or show the error message inline.

JavaScript reads both:

```
if (!response.ok) {
    const result = await response.json();
    this.showError('category-name', result.error);
    return;
}
```

`response.ok` checks the status code. `response.json()` reads the body. The conversation is complete: `fetch` sent a request, the controller processed it, and the response tells JavaScript what happened.

On success, JavaScript redirects to the list page — which triggers the full two-request cycle again.

On error (409 duplicate, 422 dependency), JavaScript reads the response and shows the message inline with `showError`.

A.17 Edit mode

Editing adds one extra request (Chapter 05 and 06). When the user clicks Edit, the URL contains `?id=2`. `checkEdit` sees it, `load(2)` fetches the record via GET, `render` fills the form. Then Save sends a PUT. Three requests total instead of two.

A.18 The complete picture

Operation	Method	Requests	Flow
Read	`GET`	2	HTML + JSON table
Create	`POST`	2	Empty form save list
Update	`PUT`	3	Form + load save list
Delete	`DELETE`	2	Confirm delete list

Every operation follows the same pattern: the HTML arrives first (the skeleton), then JavaScript does the rest (the data). The list is read-only. Every write goes through the form. One URL per entity, different HTTP methods.

Every entity — Publisher, Category, Author, Book — follows this exact flow. The files change, the pattern does not.

Appendix B

Glossary

Terms you will encounter in this book, in alphabetical order.

---

42 — The answer to the ultimate question of life, the universe, and everything. From Douglas Adams' *The Hitchhiker's Guide to the Galaxy* (1979). The joke is that nobody knows the question. In programming, it is a reminder that knowing the answer is useless if you do not understand the problem.

`.htaccess` — A configuration file read by Apache on every request to the directory where it lives. The dot prefix makes it hidden on Unix systems. In Bibliotheca, `.htaccess` contains a single rewrite rule: any URL that does not match an existing file is forwarded to `index.php?route=...`, enabling clean URLs like `/publishers` instead of `index.php?route=publishers`. Requires Apache's `mod_rewrite` module to be enabled.

API (Application Programming Interface) — A set of endpoints that a program can call to exchange data. In Bibliotheca, the files in `public/api/` are the API: JavaScript sends HTTP requests, PHP answers with JSON.

async/await — JavaScript syntax that makes asynchronous code read like sequential code. `await fetch(url)` pauses execution until the response arrives, without blocking the browser.

Böhm-Jacopini theorem — Any computable function can be written using only three control structures: sequence, selection (if/else), and iteration (for/while). No goto, no magic.

CRUD — Create, Read, Update, Delete — the four basic operations on data. Every entity in Bibliotheca implements all four through a chain of HTTP method Controller Model SQL.

Operation	HTTP	SQL
Create	POST	INSERT
Read	GET	SELECT
Update	PUT	UPDATE
Delete	DELETE	DELETE

Controller — The PHP script in `public/api/` that receives an HTTP request, calls the Model, and returns JSON. It decides *what to do* but does not know *how to query* or *how to display*.

curl — A command-line tool for making HTTP requests. Useful for testing APIs without a browser. In Bibliotheca, `curl` lets you call the API endpoints directly from the terminal: `curl http://localhost/bibliotheca/public/api/publishers.php`. The name stands for "Client URL".

DDL / DML / DQL — Three categories of SQL: - **DDL** (Data Definition Language) — `CREATE TABLE`, `ALTER TABLE` - **DML** (Data Manipulation Language) — `INSERT`, `UPDATE`, `DELETE` - **DQL** (Data Query Language) — `SELECT`

In Bibliotheca, each has its own directory under `sql/`.

Dependency injection — Passing an object to a class instead of letting the class create it internally. The Model receives the DBMS in its constructor — it does not know how the connection was opened, only that it can use it. This makes the code testable and flexible.

Document root — The directory that Apache serves to browsers. In Bibliotheca, this is `public/`. Files outside it (`src/`, `sql/`) are invisible to the web — only PHP can reach them.

DOM (Document Object Model) — The browser's in-memory representation of the HTML page. JavaScript manipulates the DOM to build tables, add rows, show errors — all without reloading the page.

DRY (Don't Repeat Yourself) — Every piece of knowledge should have a single representation. If you find yourself copying the same code, extract it.

ES6 — The 2015 edition of JavaScript that introduced classes, `const`/`let`, arrow functions, template literals, and `async/await`. Bibliotheca uses ES6 classes on the frontend.

Fail Fast — If something goes wrong, stop immediately. Do not drag an error through the code hoping it will resolve itself. In DBMS.php, `PDO::ERRMODE_EXCEPTION` enforces this: a bad query throws an exception instead of returning false.

Fail Safe — When you fail, fail securely. Never leave the system in an inconsistent or unsafe state.

`fetch` — The browser's native API for HTTP requests. Replaces the older `XMLHttpRequest`. Combined with `async/await`, it is how JavaScript talks to the backend.

Foreign key — A column that references the primary key of another table. The `publisher_id` column in `book` points to `publisher.publisher_id`. SQLite ignores foreign keys by default — `PRAGMA foreign_keys = ON` enables enforcement.

Front controller — A design pattern where every request enters the application through a single file. In Bibliotheca, `index.php` is the front controller: `.htaccess` routes all URLs to it, and it decides which page to load.

Hacker — Originally, someone who explores systems out of curiosity, taking things apart to understand how they work. The media turned the word into a synonym for criminal, but in programming culture a hacker is simply a person driven by the urge to learn by doing. That is the meaning we use in this book.

HTTP methods — The verbs of the web: GET (read), POST (create), PUT (update), DELETE (remove). Each API endpoint in Bibliotheca uses the method to decide which operation to perform.

HTTP status codes — Numbers that tell the client what happened. The ones Bibliotheca uses: - **200** — OK - **400** — Bad Request (missing or invalid input) - **404** — Not Found - **405** — Method Not Allowed - **401** — Unauthorized (not authenticated) - **409** — Conflict (duplicate record) - **422** — Unprocessable Entity (dependency error, invalid data) - **429** — Too Many Requests (rate limiting) - **500** — Internal Server Error

Index (database) — A data structure that speeds up lookups on a column, like a book index speeds up finding a topic. SQLite creates indexes automatically for primary keys, but foreign key columns need explicit `CREATE INDEX` statements. Without an index, the database must scan every row (full table scan) to find a match.

`innerHTML` **(avoid)** — A DOM property that parses a string as HTML. If the string contains user input, this is an XSS vulnerability. Bibliotheca uses `textContent` instead, which treats

everything as plain text.

JSON (JavaScript Object Notation) — A lightweight data format. The API returns JSON, JavaScript parses it with `response.json()`. It looks like this: `{"name": "Einaudi", "publisher_id": 2}`.

Junction table — A table that implements a many-to-many relationship. `book_author` connects books to authors: one book can have many authors, one author can have many books.

kebab-case — Words separated by hyphens: `book-list`, `main-header`. Used for HTML IDs, CSS classes, and git branch names.

KISS (Keep It Simple, Stupid) — Always choose the simplest solution that works. Complexity is the enemy of understanding.

`mod_rewrite` — An Apache module that rewrites URLs before they reach your code. Without it, `.htaccess` rewrite rules are silently ignored. Enable it with `sudo a2enmod rewrite` and restart Apache. It is what makes clean URLs possible — turning `/publishers` into `index.php?route=publishers` behind the scenes.

Model — A PHP class in `src/` that knows how to query one entity. It receives the DBMS via dependency injection and provides methods like `getAll`, `getById`, `insert`, `update`, `delete`.

MVC (Model-View-Controller) — An architecture that separates data (Model), presentation (View), and logic (Controller). In Bibliotheca: - **Model** — `src/*.php` - **View** — `public/pages/*.php` + `public/js/*.js` - **Controller** — `public/api/*.php`

PDO (PHP Data Objects) — PHP's database abstraction layer. It works the same way regardless of the database engine (SQLite, MySQL, PostgreSQL). Switch database by changing the connection string, nothing else.

Prepared statement — A SQL query with placeholders instead of values. The database engine separates the query from the data, making SQL injection impossible. Every query in Bibliotheca uses them.

PDO supports two styles of placeholders: - **Named** — `:id`, `:name`. Self-documenting, order-independent. - **Positional** — `?`. Shorter, but you must match the order.

We use named parameters because they are easier to read and safer to refactor. `WHERE publisher_id = :id AND status = :status` tells you what each value is. `WHERE publisher_id = ? AND status = ?` requires counting positions.

Primary key — A column that uniquely identifies each row. In Bibliotheca, it is always the first column: `publisher_id`, `book_id`, etc.

PSR-12 — A PHP coding standard that defines formatting rules: indentation, brace placement, naming. Bibliotheca follows it.

Routing — The technique of directing every request through a single entry point (`index.php`) that decides which page to load. One door, many rooms. If a route is not in the whitelist, it gets a 404. Every modern web framework uses this pattern.

Rewrite rule — An Apache directive in `.htaccess` that transforms URLs before they reach PHP. The rule `^(.*)$ index.php?route=$1` captures the URL path and passes it as a parameter to the router.

REST (Representational State Transfer) — An architectural style for web APIs, defined by Roy Fielding in 2000. The idea is simple: use what HTTP already gives you. - **Resources have URLs.** Each entity lives at its own address: `/api/books.php`, `/api/publishers.php`. - **Verbs come from HTTP.** The method *is* the action: GET reads, POST creates, PUT

updates, DELETE removes. No need for `?action=delete` in the URL. - **Responses are representations.** The server returns data (JSON), not pages. The client decides how to display it. - **Each request is self-contained.** The server does not remember previous requests. Every call carries all the information it needs.

Before REST, web services used complex protocols like SOAP — XML-heavy, with action names buried inside the message body. REST replaced all of that with what the web already had: URLs for nouns, HTTP methods for verbs, and JSON for data. Bibliotheca follows this style.

SoC (Separation of Concerns) — Each part of the code has one job. Backend serves data, frontend presents it. Model queries, Controller routes, View displays.

Soft delete vs hard delete — Two strategies for removing data. Soft delete sets a flag (`status = 0`) — the record stays in the database but appears disabled. Hard delete runs `DELETE FROM` — the record is gone. Bibliotheca uses both: the status toggle disables a record (greyed out in the list), while the Delete button removes it permanently.

SPA (Single Page Application) — A web application where the browser loads one HTML page and JavaScript handles all navigation and data updates via `fetch` calls to the API. The page never fully reloads — only the content changes. Bibliotheca follows this pattern: `index.php` serves the shell, JavaScript builds the interface.

SQL injection — An attack where user input is inserted into a SQL query, changing its meaning. Example: entering `'; DROP TABLE book; --` as a name. Prepared statements prevent this completely.

SQLite — A database engine stored in a single file. No server process, no configuration. The file `sql/bibliotheca.db` is the entire database.

`strict_types` — `declare(strict_types=1);` at the top of a PHP file enables strict type checking. Passing a string where an int is expected throws a TypeError instead of silently converting.

tail — A Unix command that shows the last lines of a file. Essential for reading logs: `sudo tail -20 /var/log/apache2/error.log` shows the 20 most recent entries. Use `tail -f` to follow a log file in real time as new entries appear.

`textContent` — A DOM property that sets or gets the text of an element. Unlike `innerHTML`, it never interprets HTML — safe by design.

Transaction — A group of database operations that must succeed together or fail together. Wrapped in `beginTransaction()` / `commit()` / `rollBack()`. In Bibliotheca, `Book::delete()` uses a transaction: deleting the `book_author` records and the `book` record must be atomic — if one fails, neither should persist.

URL rewriting — Transforming `/publishers` into `index.php?route=publishers` so the user sees clean URLs while the server uses query parameters. Done by Apache's `mod_rewrite` via `.htaccess`.

Whitelist — A list of explicitly allowed values. The router in `index.php` checks the route against a whitelist of valid pages. Anything not on the list gets a 404. Safer than a blacklist (blocking known bad values) because unknown values are rejected by default.

XSS (Cross-Site Scripting) — An attack where malicious JavaScript is injected into a page. Prevented by escaping output: `htmlspecialchars()` in PHP, `textContent` in JavaScript. Never render user input as HTML.

YAGNI (You Aren't Gonna Need It) — Don't build what you don't need yet. Solve today's problem today.

Appendix C

Bibliography

C.1 The terminal

- William Shotts, *The Linux Command Line*, Sixth Internet Edition. Free at https://linuxcommand.org

C.2 Apache

- *Apache .htaccess Tutorial* — https://httpd.apache.org/docs/2.4/howto/htaccess.html

C.3 PHP

- *PHP Manual* — https://www.php.net/manual/en/
- *PHP: The Right Way* — https://phptherightway.com
- *PHP Delusions* — https://phpdelusions.net

C.4 JavaScript, HTML, CSS

- Marijn Haverbeke, *Eloquent JavaScript*, 4th Edition. Free at https://eloquentjavascript.net
- *MDN Web Docs* — https://developer.mozilla.org

C.5 SQL and SQLite

- *SQLite Documentation* — https://sqlite.org/docs.html
- *SQLite Tutorial* — https://www.sqlitetutorial.net

C.6 HTTP

- *MDN: HTTP Guide* — https://developer.mozilla.org/en-US/docs/Web/HTTP

C.7 Git

- Scott Chacon, Ben Straub, *Pro Git*, 2nd Edition. Free at https://git-scm.com/book

C.8 The classics

- Brian W. Kernighan, Dennis M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall, 1988. You will not need C for web development, but this book will shape the way you think as a programmer.

C.9 Security

- *OWASP Top Ten* — https://owasp.org/www-project-top-ten/

Appendix D

The Apocryphal Chapter — Beyond the Launch

> And now for something completely different. — *Monty Python*

You made it through ten lessons. You built a web application from scratch — database, backend, API, frontend — with nothing between you and the code. No framework decided for you, no library hid the mechanics.

Now what?

D.1 What you actually learned

You did not just learn PHP, JavaScript, and SQL. You learned a way of thinking:

- **A request is a conversation.** The browser asks, the server answers. HTTP methods are the vocabulary. JSON is the language. One URL, different verbs — that is REST.
- **Data flows in one direction.** JavaScript calls the controller, the controller calls the model, the model calls DBMS, DBMS talks to SQLite — and back. Each layer has one job. If you respect the boundaries, the code stays simple.
- **Security is not a feature.** It is a habit. Prepared statements, `textContent`, CSRF tokens, server-side validation — these are not optional extras. They are how you write code. Each defense stands on its own.
- **The database is the truth.** Everything else — the controller, the frontend, the cache — is a representation. When in doubt, ask the database.

D.2 What Bibliotheca does not do

Bibliotheca has authentication, search, pagination, sorting, CSRF protection, and input validation. That is more than most tutorials cover. But a production application would still need:

Multi-user roles. We have a single admin. A real system needs roles (admin, editor, viewer), permissions per resource, and audit logs of who changed what.

Server-side pagination. Our pagination is client-side — all data is loaded at once, then sliced in JavaScript. With ten thousand books, you need `LIMIT` and `OFFSET` in SQL, and the API must accept `?page=3&per_page=20`.

Full-text search. Our search filters in memory. A real search needs `FTS5` in SQLite (or Elasticsearch for scale), ranking, highlighting, and fuzzy matching.

Error logging and monitoring. Our API returns appropriate status codes (400, 401, 409, 422) with error messages, but it does not log errors to a file or notify anyone. A production app

needs structured logging, alerts, and dashboards.

Deployment. Moving from `localhost` to a real server: domain names, HTTPS, firewall rules, backups, monitoring. The code is the easy part.

D.3 Frameworks — when and why

At this point, someone will tell you: "Just use Laravel" or "Just use React." And they are not wrong — for production code.

But now you know what a framework does for you:

- **A router** — you built one in `index.php`. A whitelist, a parameter, a `require`. Laravel's router does the same thing with more features.
- **An ORM** — you built models with `insert`, `getAll`, `update`, `delete`, and raw SQL. Eloquent writes the SQL for you. But when it generates a slow query, you will know why — because you understand `JOIN` and `INDEX`.
- **A template engine** — you used `createElement` and `textContent` to build the DOM. React does the same with JSX, plus reactivity. But the DOM manipulation underneath is identical.
- **Middleware** — your controllers check the HTTP method with `$_SERVER['REQUEST_METHOD']`. Express or Laravel route middleware does this declaratively.
- **A REST client** — you used `fetch` with `async/await`, built a `payload`, and chose the HTTP method. Axios or other libraries wrap the same mechanism with convenience methods.

A framework is not magic. It is someone else's `index.php`, someone else's `DBMS.php`, someone else's `Publisher.js` — packaged, tested, and documented. You can use it with confidence now, because you know what it replaces.

The danger is using a framework *without* understanding the layer below. You can configure a router without knowing what `mod_rewrite` does. You can call an ORM without knowing what a prepared statement is. But when something breaks — and it will — you will stare at the error with no idea where to look.

That is the difference between a developer and someone who uses developer tools.

D.4 The age of AI

You are learning to program in an era where machines can write code. That changes the job, not the need.

An AI can generate a CRUD endpoint in seconds. But it cannot think for you. You are the one who defines the domain, decides which entities exist, what rules govern them, and how they relate to each other. You model the problem. The machine writes code — you decide what code should be written, and whether it is right. If you give up that responsibility, you have no reason to be there. That should be unsettling enough to keep you learning. Remember this always.

The programmers who will thrive are not the ones who type fastest. They are the ones who have dominion over their code — who understand what the machine wrote, can verify it is correct, and know when to throw it away and start over. Whether you write the code yourself or an AI

writes it for you, the responsibility is yours. Dominion means understanding every line, at every level.

Learn to read, write and compute — before it's too late.

D.5 42

The code you wrote in these lessons is yours. Read it again in six months. You will understand it differently — not because the code changed, but because you did.

That is the answer.